Introduction:

What is a Database?

A database is an organized collection of data that allows for easy access, retrieval, management, and updating**.**

Why Do We Need a Database?

Imagine a pizza shop 🍕 that writes all orders on paper. It's hard to manage!
Instead, storing data in a database makes it easy to search, update, and retrieve

Types of Databases:

1)Relational Databases (RDBMS) – Data is stored in tables with relationships.

Examples: MySQL, PostgreSQL, SQL Server, Oracle
Use Case: Banking, e-commerce, social media apps (Instagram, Facebook)

2)**Non-Relational Databases (NoSQL)** – Stores data in flexible structures like documents, key-value pairs, or graphs.

- **Examples:** MongoDB, Firebase, Redis
- **Use Case:** Real-time apps, recommendation engines, IoT devices

How Databases Work in Real Life:

Instagram stores users' posts, likes, and comments in a database.
Netflix saves user preferences to recommend movies.
Online Stores (Amazon, Flipkart) store product details and user purchases.

What is SQL?

1️⃣SQL (Structured Query Language) – The Language of Databases
SQL is a language used to interact with relational databases. It helps in:
✔️ Adding new data (INSERT)
✔️ Retrieving data (SELECT)
✔️ Updating existing data (UPDATE)
✔️ Deleting unnecessary data (DELETE)

Why is SQL Important?

Used in Web & Mobile Apps (Instagram, YouTube, Amazon).
Helps in Data Analysis (Companies use SQL to analyze sales, trends).
Powers AI & Machine Learning (Data stored in databases is used for AI training)

SQL in Real Life:

💡 Example: How Instagram Uses SQL

When you post a photo, SQL is used to store the image details in a database.
When you like a post, SQL updates the database with your like.
When you search for a friend, SQL retrieves matching names from the database.

How to Create a Database & Table in SQL:

1️⃣ Creating a Database
A database is a structured collection of data that allows efficient storage, retrieval, and management.

SQL Command to Create a Database:

```
CREATE DATABASE my_database;
```

CREATE DATABASE → This SQL command is used to create a new database.

my_database → The name of the database (you can replace it with any name you want).

Using the Database:
Before creating tables, you need to select the database

Command to use:

```
USE my_database;
```

This sets the active database where tables will be created.

2️⃣ Creating a Table in SQL
A table is where actual data is stored in a structured format using rows and columns.

SQL Command to Create a Table:

```
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  age INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Explanation of the Columns:

id INT PRIMARY KEY AUTO_INCREMENT → Unique ID for each user, auto-incrementing.

name VARCHAR(100) NOT NULL → Stores the user's name, up to 100 characters, and cannot be empty.

email VARCHAR(100) UNIQUE NOT NULL → Stores email, must be unique, and cannot be empty.

age INT → Stores user's age as an integer.

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP → Stores the time the record was created automatically.

SQL Data Types:

1 **Numeric Data Types** (For storing numbers)

INT → Whole numbers (e.g., 1, 100, -50)

TINYINT → Small numbers (-128 to 127)

SMALLINT → Medium-sized numbers (-32,768 to 32,767)

BIGINT → Large numbers

DECIMAL(p, s) → Fixed decimal numbers (e.g., 99.99)

FLOAT → Floating-point numbers (e.g., 3.14, 0.0005)

2 **String Data Types** (For storing text)

CHAR(n) → Fixed-length text (e.g., 'YES', 'NO')

VARCHAR(n) → Variable-length text (e.g., 'Hello World')

TEXT → Large text data

3 Date & Time Data Types (For storing dates and times)

DATE → Stores date (YYYY-MM-DD)

TIME → Stores time (HH:MM:SS)

DATETIME → Stores date & time
(YYYY-MM-DD HH:MM:SS)

TIMESTAMP → Auto-updating date & time

4⃣ **Boolean Data Type** (For storing true/false values)

BOOLEAN → Stores TRUE (1) or FALSE (0)

5⃣ **Special Data Types** (For special data storage)

BLOB → Stores binary data (e.g., images, files)

JSON → Stores structured JSON data

Insertion of data in a Table:

In SQL, the INSERT statement is used to add new records (rows) into a table. Below are the different methods and best practices for inserting data into an SQL database.

1. Basic Syntax of INSERT Statement
The general syntax for inserting data into a table is:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

table_name: The name of the table where data will be inserted.
column1, column2, column3: The specific columns in which the data will be inserted.
value1, value2, value3: The values corresponding to each column.
Example:
Assume we have a students table:

```
CREATE TABLE students (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   grade VARCHAR(10)
);
```

To insert a new student record:

```
INSERT INTO students (id, name, age, grade)
VALUES (1, 'John Doe', 18, 'A');
```

2. Inserting Data into All Columns
If you are inserting values for all columns in order, you can omit the column names:

```
INSERT INTO students
VALUES (2, 'Jane Smith', 19, 'B');
```

⚠ **Caution: The order of values must match the column order in the table.**

3. Inserting Multiple Rows at Once
SQL allows inserting multiple records in a single query:

```
INSERT INTO students (id, name, age, grade)
```

```
VALUES
   (3, 'Alice Brown', 20, 'A'),
   (4, 'Bob White', 22, 'C'),
   (5, 'Charlie Green', 21, 'B');
```
✅ Benefits: Reduces the number of queries, improves performance.

4. Using DEFAULT Values
If a column has a default value and you don’t want to specify it:

```
INSERT INTO students (id, name, age)
VALUES (6, 'Emma Wilson', 20);
```
Here, the grade column will use its default value if defined.

5. Inserting Data from Another Table
To copy data from one table to another:

```
INSERT INTO new_students (id, name, age, grade)
SELECT id, name, age, grade FROM students WHERE grade = 'A';
```
6. Using AUTO_INCREMENT for Primary Key
If the primary key column is set to AUTO_INCREMENT, you don’t need to insert its value:

```
CREATE TABLE employees (
   emp_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   salary DECIMAL(10,2)
);

INSERT INTO employees (name, salary)
VALUES ('David Miller', 50000.00);
```
The emp_id will be assigned automatically.

7. Handling NULL Values
If a column allows NULL, you can insert a NULL value:

```
INSERT INTO students (id, name, age, grade)
VALUES (7, 'Olivia Davis', NULL, NULL);
```
8. Using INSERT with SELECT
You can insert data from a SELECT query:

```
INSERT INTO employees (name, salary)
SELECT name, 60000 FROM students WHERE age > 21;
```
9. Checking Data After Insertion
To verify inserted data:

```
SELECT * FROM students;
```

Deleting and Updating Data in a Table:

In relational databases like MySQL, PostgreSQL, SQLite, or SQL Server, we often need to modify or remove existing records in a table. SQL provides DELETE and UPDATE statements for these operations.

1. Deleting Data from a Table:
The DELETE statement is used to remove records from a table.

Syntax:
DELETE FROM table_name WHERE condition;

•table_name: The name of the table from which you want to delete data.
•WHERE condition: Specifies which records to delete. If omitted, ALL records will be deleted.

Examples
a) Delete a Specific Row:

DELETE FROM employees WHERE employee_id = 101;

b) Delete Multiple Rows:
DELETE FROM employees WHERE department = 'HR';

Deletes all employees from the HR department.

c) Delete All Records (Use with Caution)
DELETE FROM employees;

d) Truncate vs Delete
TRUNCATE TABLE employees;

•TRUNCATE removes all rows faster than DELETE but cannot be rolled back in some databases.
•DELETE can be rolled back if used inside a transaction.

2. **Updating Data in a Table:**
The UPDATE statement is used to modify existing records in a table.

Syntax:
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;

•table_name: The name of the table.
•SET: Specifies the column(s) to update with new values.
•WHERE: Specifies which records to update

Examples

a) **Update a Single Row:**

```
UPDATE employees
SET salary = 60000
WHERE employee_id = 101;
```

Updates the salary of the employee with ID 101 to 60,000.

b) **Update Multiple Rows:**

```
UPDATE employees
SET department = 'Finance'
WHERE department = 'Accounts';
```

Changes all employees in the "Accounts" department to "Finance".
c) **Update All Rows (Use with Caution):**
UPDATE employees SET salary = salary * 1.10;
Increases the salary of all employees by 10%.

Modifying Table Structure in SQL:

When working with databases, you may need to add, rename, or delete columns in a table. These operations can be performed using the ALTER TABLE statement.

1**. Adding a Column to a Table:**

You can add a new column to an existing table using the ADD COLUMN command.

Syntax:
ALTER TABLE table_name ADD COLUMN column_name data_type [constraints];

•table_name: Name of the table where the column will be added.
•column_name: Name of the new column.
•data_type: Data type of the column (INT, VARCHAR(255), DATE, etc.).
•[constraints]: Optional constraints like NOT NULL, UNIQUE, DEFAULT, etc.

Examples
a) **Add a Column Without Constraints:**
ALTER TABLE employees ADD COLUMN age INT;

b) Add a Column With Default Value:

ALTER TABLE employees ADD COLUMN joining_date DATE DEFAULT '2024-01-01';

Adds a joining_date column with a default value.

c)Add Multiple Columns: de

```
ALTER TABLE employees
ADD COLUMN address VARCHAR(255),
ADD COLUMN phone_number VARCHAR(15);
```

Adds two columns, address and phone_number, to the employees table.

2. **Renaming a Column:**
To rename an existing column, the syntax varies depending on the database.
Syntax
For MySQL, PostgreSQL (Version 9+), and MariaDB

```
ALTER TABLE table_name RENAME COLUMN old_column_name TO new_column_name;
```

For SQL Server:

```
EXEC sp_rename 'table_name.old_column_name', 'new_column_name', 'COLUMN';
```

Examples:

```
ALTER TABLE employees RENAME COLUMN age TO employee_age;
```

Renames the age column to employee_age in the employees table.

```
EXEC sp_rename 'employees.phone_number', 'contact_number', 'COLUMN';
```

Renames the phone_number column to contact_number in SQL Server.

3. Deleting a Column from a Table
You can remove an existing column using the DROP COLUMN command.

Syntax:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

•table_name: Name of the table.
•column_name: The column to be removed.

Examples:
a) Delete a Single Column

```
ALTER TABLE employees DROP COLUMN age;
```

Removes the age column from the employees table.

b) Delete Multiple Columns:

```
ALTER TABLE employees
DROP COLUMN address,
DROP COLUMN phone_number;
```

Deletes both address and phone_number columns.

⚠ Note: Some databases (like MySQL) do not support dropping multiple columns in one statement. You may need to run separate ALTER TABLE statements.

4. Modifying a Column's Data Type:

You may need to change the data type of an existing column.
a) **Changing Column Type in MySQL:**
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(10,2);

5. **Renaming a Table:**
If you need to rename an entire table:

syntax:
ALTER TABLE old_table_name RENAME TO new_table_name;

•old_table_name: The current name of the table.
•new_table_name: The new name you want to assign.
Example:
ALTER TABLE employees RENAME TO staff;

Renames the employees table to staff.

Primary Keys & Foreign Keys:
Primary Key

A column (or set of columns) that uniquely identifies each row in a table.

Cannot contain NULL or duplicate values.

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100)
);
```

Foreign Key:

A column that links two tables together by referencing the primary key in another table.

Ensures referential integrity.

```
CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

Understanding Relationships:
1. One-to-One

Example: One person has one passport.

```
CREATE TABLE Person (
  person_id INT PRIMARY KEY,
  name VARCHAR(100)
);

CREATE TABLE Passport (
  passport_id INT PRIMARY KEY,
  person_id INT UNIQUE,
  FOREIGN KEY (person_id) REFERENCES Person(person_id)
);
```

2)One-to-Many

Example: One teacher teaches many students.

3. Many-to-Many

Example: A student can enroll in many courses, and a course can have many students. Use a junction table.

```
CREATE TABLE Course (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100)
);

CREATE TABLE Student_Course (
  student_id INT,
  course_id INT,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Course(course_id)
);
```

SQL Joins :

Joins in SQL are used to combine rows from two or more tables, based on a related column between them.

1) INNER JOIN

Returns only matching rows from both tables.

```
SELECT A.*, B.*
FROM TableA A
INNER JOIN TableB B ON A.id = B.a_id;
```

Use Case: When you want data present in both tables.

2)LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table, and matching rows from the right table.

If there's no match, NULLs are returned for right table columns.

```
SELECT A.*, B.*
FROM TableA A
LEFT JOIN TableB B ON A.id = B.a_id;
```

Use Case: When you need all data from left table, regardless of match.

3)RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table, and matching rows from the left table.

```
SELECT A.*, B.*
FROM TableA A
RIGHT JOIN TableB B ON A.id = B.a_id;
```

Use Case: When you need all data from right table, regardless of match.

4)FULL JOIN (FULL OUTER JOIN)

Returns all rows from both tables.

Where there is no match, NULLs are returned

```
SELECT A.*, B.*
FROM TableA A
FULL JOIN TableB B ON A.id = B.a_id;
```

Use Case: When you want everything, whether matched or not.

5)CROSS JOIN
Returns cartesian product: every row from TableA paired with every row from TableB.

```
SELECT A.*, B.*
FROM TableA A
CROSS JOIN TableB B;
```

Use Case: Rarely used. Helpful in generating combinations.

6)SELF JOIN
A table is joined with itself.

```
SELECT A.name, B.name
FROM Employees A
JOIN Employees B ON A.manager_id = B.emp_id;
```

Use Case: Hierarchical data, like employees and their managers.

Subqueries & Nested Queries:

Subquery: A query within another query.

Used in WHERE, FROM, or SELECT.

Example: Get students enrolled in more than 1 course.

```
SELECT name FROM Students
WHERE student_id IN (
  SELECT student_id
  FROM Student_Course
  GROUP BY student_id
  HAVING COUNT(course_id) > 1
);
```

UNION & INTERSECT:

UNION: Combines rows from two queries, removes duplicates.

```
SELECT name FROM Students
UNION
SELECT name FROM Alumni;
```

UNION ALL: Includes duplicates.

INTERSECT: Returns only rows present in both queries (Not all DBMS support it).

```
SELECT name FROM Students
INTERSECT
SELECT name FROM TopPerformers;
```

Aggregations (SUM, COUNT, AVG, MIN, MAX):

Aggregate functions perform a calculation on a set of values and return a single value.

Common Aggregates:
1)SUM() – Returns the total sum of a numeric column.
SELECT SUM(salary) FROM employees;

2)COUNT() – Returns the number of rows (or non-NULL values).

3)AVG() – Returns the average value.
SELECT AVG(salary) FROM employees;

4)MIN() – Returns the smallest value.
SELECT MIN(salary) FROM employees;

5)MAX() – Returns the largest value.
SELECT MAX(salary) FROM employees;

Note: These can be used with WHERE, but not directly with GROUP BY unless aggregating over groups.

Grouping Data (GROUP BY, HAVING)

GROUP BY
Used to group rows with the same values into summary rows.

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

HAVING
Used to filter groups based on aggregate values (like WHERE but for groups).

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Note: HAVING comes after GROUP BY and works on aggregate results.

Views & Materialized Views:

Views:
A view is a virtual table created from a SELECT query.

```
CREATE VIEW high_earners AS
SELECT * FROM employees WHERE salary > 80000;
```

Always shows the latest data.

Cannot always be updated (depends on the query).
Materialized Views:
Stores the result of the query physically.

```
CREATE MATERIALIZED VIEW dept_salary_summary AS
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

1)Can be refreshed manually or automatically.

2)Useful for performance when dealing with expensive queries.

Stored Procedures:

A stored procedure is a precompiled collection of SQL statements stored in the database.

Syntax (MySQL example):

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(50))
BEGIN
  SELECT * FROM employees WHERE department = dept_name;
END //

DELIMITER ;
```

Calling a procedure:

```
CALL GetEmployeesByDept('IT');
```

1)Can take input, output, or inout parameters.

2)Great for encapsulating business logic.

Normalization in DBMS:

Normalization is the process of organizing data in a database to reduce data redundancy and improve data integrity. It involves dividing large tables into smaller, related tables and defining relationships between them.

Objectives of Normalization:
Minimize data redundancy (avoid duplicate data)

Ensure data consistency

Simplify data structures

Facilitate easier database maintenance

Improve data integrity and update efficiency

Types (Forms) of Normalization:

Normalization is done in stages, called Normal Forms (NF). Each form has specific rules and builds on the previous one.

1) First Normal Form (1NF)
Rule:

Atomicity: All values in a column must be atomic (indivisible).

Each column should contain only one value per record.

No repeating groups or arrays.

Example:

Before 1NF:

StudentID	Name	Subjects
1	Alice	Math, Science
2	Bob	English, History

After 1NF:

StudentID	Name	Subject
1	Alice	Math
1	Alice	Science
2	Bob	English
2	Bob	History

2)Second Normal Form (2NF)
Rule:

Must be in 1NF

No partial dependency (i.e., non-prime attribute should not depend on part of the primary key)

Partial Dependency: Occurs when a non-key attribute is dependent on part of a composite primary key.

Example:

Before 2NF:

StudentID	CourseID	StudentName	CourseName
1	C1	Alice	Math
2	C2	Bob	History

Here, StudentName depends only on StudentID and CourseName only on CourseID.

After 2NF:

Student Table:

StudentID	StudentName
1	Alice
2	Bob

Course Table:

CourseID	CourseName
C1	Math
C2	History

Enrollment Table:

StudentID	CourseID
1	C1
2	C2

3)Third Normal Form (3NF)
Rule:

Must be in 2NF

No transitive dependency (i.e., non-prime attribute should not depend on another non-prime attribute)

Transitive Dependency: A → B and B → C implies A → C (indirect dependency)

Example:

Before 3NF:

StudentID	Name	DeptID	DeptName
1	Alice	D1	Science
2	Bob	D2	Arts

Here, DeptName depends on DeptID, which in turn depends on StudentID.

After 3NF:

Student Table:

StudentID	Name	DeptID
1	Alice	D1
2	Bob	D2

Department Table:

DeptID	DeptName
D1	Science
D2	Arts

Boyce-Codd Normal Form (BCNF)
Rule:

Stricter version of 3NF

Every determinant must be a candidate key

Used when 3NF fails due to more than one candidate key and dependencies still exist.

4NF (Fourth Normal Form)
Rule:

Must be in BCNF

No multi-valued dependencies

Example: A teacher can teach multiple subjects and be assigned to multiple rooms. These are independent multi-valued facts and need to be separated.

5NF (Fifth Normal Form or PJNF)
Rule:

Must be in 4NF

No join dependency

Data should not be reconstructed by joining multiple tables unnecessarily

Advantages of Normalization:

Eliminates redundant data

Ensures data dependencies make sense

Improves data integrity and consistency

Easier to update, insert, or delete records

Disadvantages of Normalization:

More complex queries due to multiple tables

Performance issues in read-heavy databases

Might require more joins and impact speed

SQL Injection:
SQL Injection is a code injection technique where malicious SQL statements are inserted into an entry field for execution. It allows attackers to interfere with the queries that an application makes to its database.

Goals of SQL Injection:

Bypass authentication

Extract data (usernames, passwords, credit cards, etc.)

Modify/delete data

Execute admin operations on the database

Take control of the server in extreme cases

How SQL Injection Works:
Imagine a vulnerable login query like this:
SELECT * FROM users WHERE username = 'user' AND password = 'pass';

If the attacker enters:

Username: ' OR '1'='1

Password: ' OR '1'='1

The resulting query becomes:
SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '' OR '1'='1';

This always returns true, so the attacker logs in without valid credentials.